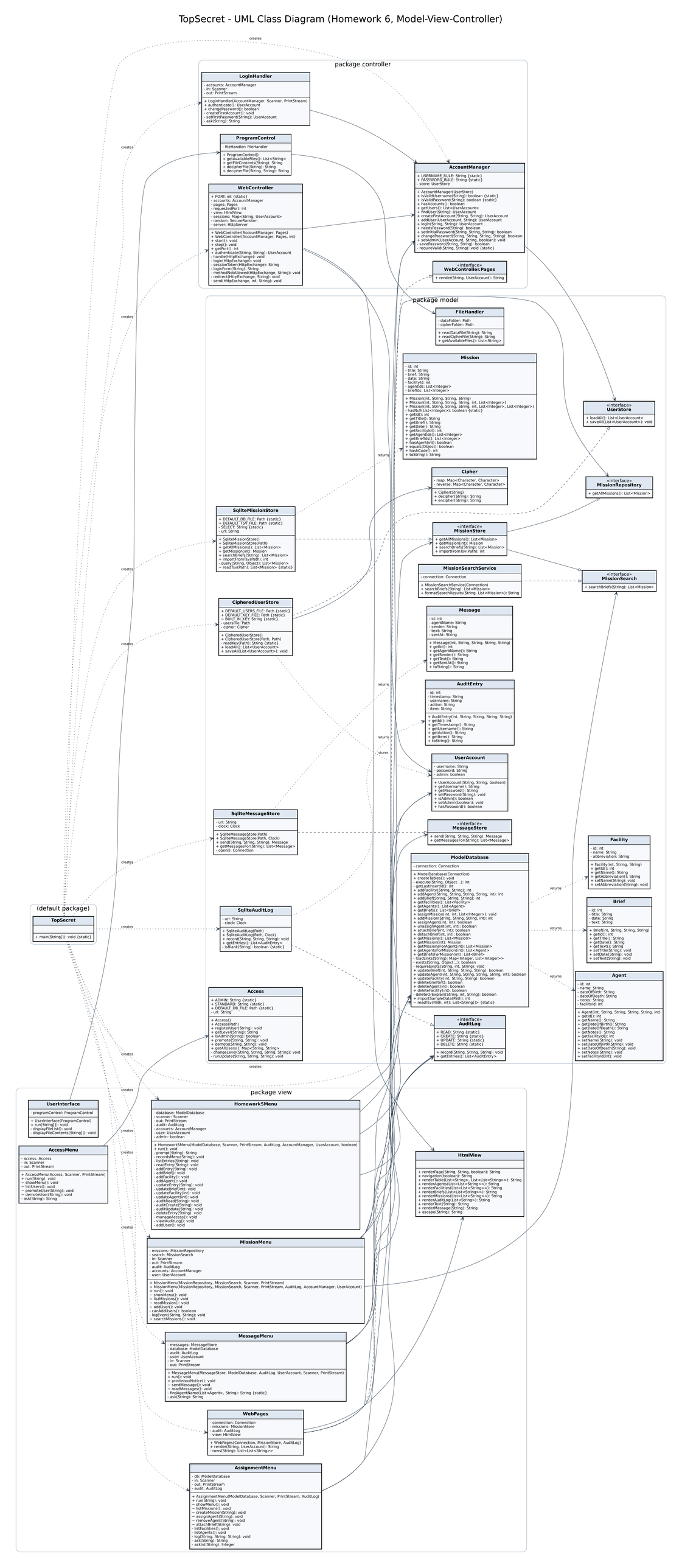
TopSecret - UML Class Diagram (Homework 6, Model-View-Controller)
package controller
package model
package view
(default package)
creates
returns
stores
LoginHandler
- accounts: AccountManager
- in: Scanner
- out: PrintStream
+ LoginHandler(AccountManager, Scanner, PrintStream)
+ authenticate(): UserAccount
+ changePassword(): boolean
- createFirstAccount(): void
- setFirstPassword(String): UserAccount
- ask(String): String
ProgramControl
- fileHandler: FileHandler
+ ProgramControl()
+ getAvailableFiles(): List<String>
+ getFileContents(String): String
+ decipherFile(String): String
+ decipherFile(String, String): String
WebController
+ PORT: int {static}
- accounts: AccountManager
- pages: Pages
- httpPort: int
- view: HtmlView
- sessions: Map<String, UserAccount>
- random: SecureRandom
- server: HttpServer
+ WebController(AccountManager, Pages)
+ WebController(AccountManager, Pages, int)
+ start(): void
+ stop(): void
+ getPort(): int
+ authenticate(String, String): UserAccount
- handle(HttpExchange): void
- login(HttpExchange): void
- sessionToken(HttpExchange): String
- loginForm(String): String
- methodNotAllowed(HttpExchange, String): void
- redirect(HttpExchange, String): void
- send(HttpExchange, int, String): void
AccountManager
+ USERNAME_RULE: String {static}
+ PASSWORD_RULE: String {static}
- store: UserStore
+ AccountManager(UserStore)
+ isValidUsername(String): boolean {static}
+ isValidPassword(String): boolean {static}
+ hasAccounts(): boolean
+ getUsers(): List<UserAccount>
+ findUser(String): UserAccount
+ createFirstAccount(String, String): UserAccount
+ addUser(UserAccount, String): UserAccount
+ login(String, String): UserAccount
+ needsPassword(String): boolean
+ setInitialPassword(String, String, String): boolean
+ changePassword(String, String, String, String): boolean
+ setAdmin(UserAccount, String, boolean): void
- savePassword(String, String): boolean
- requireValid(String, String): void {static}
«interface»
WebController.Pages
+ render(String, UserAccount): String
FileHandler
- dataFolder: Path
- cipherFolder: Path
+ readDataFile(String): String
+ readCipherFile(String): String
+ getAvailableFiles(): List<String>
Mission
- id: int
- title: String
- brief: String
- date: String
- facilityId: int
- agentIds: List<Integer>
- briefIds: List<Integer>
+ Mission(int, String, String, String)
+ Mission(int, String, String, String, int, List<Integer>)
+ Mission(int, String, String, String, int, List<Integer>, List<Integer>)
- hasNull(List<Integer>): boolean {static}
+ getId(): int
+ getTitle(): String
+ getBrief(): String
+ getDate(): String
+ getFacilityId(): int
+ getAgentIds(): List<Integer>
+ getBriefIds(): List<Integer>
+ hasAgent(int): boolean
+ equals(Object): boolean
+ hashCode(): int
+ toString(): String
«interface»
UserStore
+ loadAll(): List<UserAccount>
+ saveAll(List<UserAccount>): void
Cipher
- map: Map<Character, Character>
- reverse: Map<Character, Character>
+ Cipher(String)
+ decipher(String): String
+ encipher(String): String
«interface»
MissionRepository
+ getAllMissions(): List<Mission>
SqliteMissionStore
+ DEFAULT_DB_FILE: Path {static}
+ DEFAULT_TSV_FILE: Path {static}
- SELECT: String {static}
- url: String
+ SqliteMissionStore()
+ SqliteMissionStore(Path)
+ getAllMissions(): List<Mission>
+ getMission(int): Mission
+ searchBriefs(String): List<Mission>
+ importFromTsv(Path): int
- query(String, Object): List<Mission>
- readTsv(Path): List<Mission> {static}
«interface»
MissionStore
+ getAllMissions(): List<Mission>
+ getMission(int): Mission
+ searchBriefs(String): List<Mission>
+ importFromTsv(Path): int
MissionSearchService
- connection: Connection
+ MissionSearchService(Connection)
+ searchBriefs(String): List<Mission>
+ formatSearchResults(String, List<Mission>): String
«interface»
MissionSearch
+ searchBriefs(String): List<Mission>
CipheredUserStore
+ DEFAULT_USERS_FILE: Path {static}
+ DEFAULT_KEY_FILE: Path {static}
- BUILT_IN_KEY: String {static}
- usersFile: Path
- cipher: Cipher
+ CipheredUserStore()
+ CipheredUserStore(Path, Path)
- readKey(Path): String {static}
+ loadAll(): List<UserAccount>
+ saveAll(List<UserAccount>): void
Message
- id: int
- agentName: String
- sender: String
- text: String
- sentAt: String
+ Message(int, String, String, String, String)
+ getId(): int
+ getAgentName(): String
+ getSender(): String
+ getText(): String
+ getSentAt(): String
+ toString(): String
AuditEntry
- id: int
- timestamp: String
- username: String
- action: String
- item: String
+ AuditEntry(int, String, String, String, String)
+ getId(): int
+ getTimestamp(): String
+ getUsername(): String
+ getAction(): String
+ getItem(): String
+ toString(): String
UserAccount
- username: String
- password: String
- admin: boolean
+ UserAccount(String, String, boolean)
+ getUsername(): String
+ getPassword(): String
+ setPassword(String): void
+ isAdmin(): boolean
+ setAdmin(boolean): void
+ hasPassword(): boolean
SqliteMessageStore
- url: String
- clock: Clock
+ SqliteMessageStore(Path)
+ SqliteMessageStore(Path, Clock)
+ send(String, String, String): Message
+ getMessagesFor(String): List<Message>
- open(): Connection
«interface»
MessageStore
+ send(String, String, String): Message
+ getMessagesFor(String): List<Message>
ModelDatabase
- connection: Connection
+ ModelDatabase(Connection)
+ createTables(): void
- execute(String, Object...): int
- queryOne(String, Object...): Optional<Map<String, Object>>
+ addFacility(String, String): int
+ addAgent(String, String, String, String, int): int
+ addBrief(String, String, String): int
+ getFacilities(): List<Facility>
+ getAgents(): List<Agent>
+ getBriefs(): List<Brief>
+ assignMission(int, int, List<Integer>): void
+ addMission(String, String, String, int): int
+ assignAgent(int, int): boolean
+ unassignAgent(int, int): boolean
+ attachBrief(int, int): boolean
+ detachBrief(int, int): boolean
+ getMissions(): List<Mission>
+ getMission(int): Mission
+ getMissionsForAgent(int): List<Mission>
+ getAgentsForMission(int): List<Agent>
+ getBriefsForMission(int): List<Brief>
- loadLinks(String): Map<Integer, List<Integer>>
- exists(String, Object...): boolean
- requireExists(String, int, String): void
+ updateBrief(int, String, String, String): boolean
+ updateAgent(int, String, String, String, int): boolean
+ updateFacility(int, String, String): boolean
+ deleteBrief(int): boolean
+ deleteAgent(int): boolean
+ deleteFacility(int): boolean
- deleteOrExplain(String, int, String): boolean
+ importSampleData(Path): int
- readTsv(Path, int): List<String[]> {static}
Facility
- id: int
- name: String
- abbreviation: String
+ Facility(int, String, String)
+ getId(): int
+ getName(): String
+ getAbbreviation(): String
+ setName(String): void
+ setAbbreviation(String): void
Brief
- id: int
- title: String
- date: String
- text: String
+ Brief(int, String, String, String)
+ getId(): int
+ getTitle(): String
+ getDate(): String
+ getText(): String
+ setTitle(String): void
+ setDate(String): void
+ setText(String): void
Agent
- id: int
- name: String
- dateOfBirth: String
- dateOfDeath: String
- notes: String
- facilityId: int
+ Agent(int, String, String, String, String, int)
+ getId(): int
+ getName(): String
+ getDateOfBirth(): String
+ getDateOfDeath(): String
+ getNotes(): String
+ getFacilityId(): int
+ setName(String): void
+ setDateOfBirth(String): void
+ setDateOfDeath(String): void
+ setNotes(String): void
+ setFacilityId(int): void
SqliteAuditLog
- url: String
- clock: Clock
+ SqliteAuditLog(Path)
+ SqliteAuditLog(Path, Clock)
+ record(String, String, String): void
+ getEntries(): List<AuditEntry>
- isBlank(String): boolean {static}
«interface»
AuditLog
+ READ: String {static}
+ CREATE: String {static}
+ UPDATE: String {static}
+ DELETE: String {static}
+ record(String, String, String): void
+ getEntries(): List<AuditEntry>
Access
+ ADMIN: String {static}
+ STANDARD: String {static}
+ DEFAULT_DB_FILE: Path {static}
- url: String
+ Access()
+ Access(Path)
+ registerUser(String): void
+ getLevel(String): String
+ isAdmin(String): boolean
+ promote(String, String): void
+ demote(String, String): void
+ getAllUsers(): Map<String, String>
- changeLevel(String, String, String, String): void
- runUpdate(String, String, String): void
TopSecret
+ main(String[]): void {static}
UserInterface
- programControl: ProgramControl
+ UserInterface(ProgramControl)
+ run(String[]): void
- displayFileList(): void
- displayFileContents(String[]): void
AccessMenu
- access: Access
- in: Scanner
- out: PrintStream
+ AccessMenu(Access, Scanner, PrintStream)
+ run(String): void
~ showMenu(): void
~ listUsers(): void
~ promoteUser(String): void
~ demoteUser(String): void
- ask(String): String
Homework5Menu
- database: ModelDatabase
- scanner: Scanner
- out: PrintStream
- audit: AuditLog
- accounts: AccountManager
- user: UserAccount
- admin: boolean
+ Homework5Menu(ModelDatabase, Scanner, PrintStream, AuditLog, AccountManager, UserAccount, boolean)
+ run(): void
- prompt(String): String
- recordsMenu(String): void
- listEntries(String): void
- readEntry(String): void
- addEntry(String): void
- addBrief(): void
- addFacility(): void
- addAgent(): void
- updateEntry(String): void
- updateFacility(int): void
- updateBrief(int): void
- updateAgent(int): void
- auditRead(String): void
- auditCreate(String): void
- auditUpdate(String): void
- deleteEntry(String): void
- manageAccess(): void
- viewAuditLog(): void
- addUser(): void
MissionMenu
- missions: MissionRepository
- search: MissionSearch
- in: Scanner
- out: PrintStream
- audit: AuditLog
- accounts: AccountManager
- user: UserAccount
+ MissionMenu(MissionRepository, MissionSearch, Scanner, PrintStream)
+ MissionMenu(MissionRepository, MissionSearch, Scanner, PrintStream, AuditLog, AccountManager, UserAccount)
+ run(): void
~ showMenu(): void
~ listMissions(): void
~ readMission(): void
~ addUser(): void
- canAddUser(): boolean
- logEvent(String, String): void
~ searchMission(): void
MessageMenu
- messages: MessageStore
- database: ModelDatabase
- audit: AuditLog
- user: UserAccount
- in: Scanner
- out: PrintStream
+ MessageMenu(MessageStore, ModelDatabase, AuditLog, UserAccount, Scanner, PrintStream)
+ run(): void
+ printInboxNotice(): void
~ sendMessage(): void
~ readMessage(): void
- findAgentName(List<Agent>, String): String {static}
- ask(String): String
WebPages
- connection: Connection
- missions: MissionStore
- audit: AuditLog
- view: HtmlView
+ WebPages(Connection, MissionStore, AuditLog)
+ render(String, UserAccount): String
- rows(String): List<List<String>>
HtmlView
+ renderPage(String, String, boolean): String
+ navigation(boolean): String
+ renderTable(List<String>, List<List<String>>): String
+ renderAgents(List<List<String>>): String
+ renderFacilities(List<List<String>>): String
+ renderBriefs(List<List<String>>): String
+ renderMissions(List<List<String>>): String
+ renderAuditLog(List<String>): String
+ renderText(String): String
+ renderMessages(String): String
+ escape(String): String
AssignmentMenu
- db: ModelDatabase
- in: Scanner
- out: PrintStream
- audit: AuditLog
+ AssignmentMenu(ModelDatabase, Scanner, PrintStream, AuditLog)
+ run(String): void
~ showMenu(): void
~ listMissions(): void
~ createMission(String): void
~ assignAgent(String): void
~ removeAgent(String): void
~ attachBrief(String): void
- listFacilities(): void
- listAgents(): void
- log(String, String, String): void
- ask(String): String
- askInt(String): Integer